CS781 Capstone: Methods Overview

Legal NLP on Indian legal documents with our own Phase-1 language models: what each phase uses, and how every fine-tuning technique works

Status of this document (29 Oct 2026). **Done**: Phase 1 (four models trained from scratch). **Implemented**: Phase 2 pipeline (package `p2lner`, Kaggle notebooks v1/v2), tested on the real Phase-1 checkpoints with synthetic data. **Planned**: Phases 3–7. Their sections describe the intended design (head, loss, pooling, techniques), not finished work. No leaderboard numbers are reported here. Results belong in the report, taken from `RESULTS.csv` and Codabench.

1. The big picture

Every phase follows the same recipe. One shared **backbone** (a Phase-1 model) is adapted with a **fine-tuning technique** and topped with a small **task module**. Only the task module changes much from phase to phase: how the backbone output is read (pooling), the head, the loss and the decoding.

Stage	Shared across phases	Changes per phase
Text → tokens	Phase-1 byte-level BPE (32,000 vocab) with character offsets	Input format: whole document, sentence list, query + candidates
Backbone	A / A_v2 / B-encoder / C / D loaded from `model_final.pt`; causal or bidirectional attention	Which models are compared; context length / window
Adaptation	Full, partial, LoRA, LoRMA, adapters, BitFit, prompt tuning, KD (`peft.py`, `model.py`)	Which ≥3 techniques are run (LORMA always; KD always)
Task module	–	Pooling (token / sentence / document), head, loss, decoding
Long documents	Sliding windows; scores averaged over overlapping windows	How windows are combined (per token vs. per document)
Evaluation / output	Fixed dev split, error analysis, `prediction.json` in `submission.zip`	Metric and output schema

Rules that hold in every phase (2–7):

- Start from our own Phase-1 models. No other pretrained weights and no external LLM APIs. Only the official competition data.
- Use all four Phase-1 architectures. The best four submissions must involve **different** Phase-1 models.
- At least 4 methodologically different experiments: ≥3 different fine-tuning / PEFT techniques (**LORMA compulsory**) and ≥1 **knowledge distillation** experiment. Changing only hyper-parameters does not count.
- Submissions: minimum 4–6 valid ones (the document states both), at most 10 per day and 40 per phase. Identical resubmissions do not count.

2. Phase 1: the foundation models

Done. Four architectures trained from scratch on the official legal corpus (Acts 20 %, Supreme Court 35 %, High Courts 25 %, Tribunals 20 %; about 500 M tokens tokenised, 400 M-token training budget, T4 GPU, about 4–10 h each). Primary metric: BERTScore F1 of 128-word continuations of 512-word prompts.

Model	Architecture	Size	Pretraining objective	Local ROUGE-L / BLEU-4	Role from Phase 2 on
A	Decoder-only LLaMA-style: RMSNorm, RoPE, SwiGLU, grouped-query attention; 12 × 640	≈72 M	Causal LM	0.287 / 17.7	baseline / KD student candidate
A_v2	Same family, 11 layers × 768, 4 KV heads	≈94 M	Causal LM	**0.289** / 17.8	the "A" model used in Phases 2+
B	Encoder-decoder (T5-like): 6 + 6 layers × 576, cross-attention	74 M (encoder 42 M)	Prompt → continuation (seq2seq)	0.177 / 3.4	encoder for classification; full model for Phase 6

Model	Architecture	Size	Pretraining objective	Local ROUGE-L / BLEU-4	Role from Phase 2 on
C	Hybrid UniLM-style shared stack, 10 × 640, segment embeddings	≈70 M	Prefix-LM + 20 % MLM samples	0.274 / 16.2	natural bidirectional encoder (strongest NER candidate)
D	Efficient decoder: sliding-window attention (256) + multi-query attention; 12 × 640	≈71 M	Causal LM, long context (2048)	0.274 / 16.3	long-document model

Why B scored low in Phase 1. Only one third of each 1,152-token sample was a prediction target, so B learned from about 133 M supervised tokens versus 400 M for A. Also, its decoder starts from <bos> and cross-attention carries no position information, so continuations drift away from the prompt's last sentence. Both are decoder problems; the encoder used for classification is unaffected.

3. Phase by phase

For each phase: the data, what goes in and out, how the task module is built, and **what is different** from the other phases. The backbone (Section 2) and techniques (Section 4) are shared.

Phase 2: Legal Named Entity Recognition (L-NER) [Implemented]

Data	Full judgments; fold1 + fold2 annotated (train), fold3 (test). No dev split: a fixed 15 % of documents is held out.
Input → output	Document text → list of [start_char, end_char, label] with 12 labels (APP, RESP, A.COUNSEL, R.COUNSEL, JUDGE, WIT, AUTH, COURT, STAT, PREC, DATE, CASENO).
Metric	Strict macro-F1: start, end and label must all match exactly (one character off counts as wrong).
Task module	Token level. Each token's hidden state → Linear(d → 25 BIO tags) or a linear-chain **CRF**. Token CE (optional class weights / focal) or CRF NLL. BIO-constrained Viterbi decoding, then boundary characters that never start or end a gold entity are trimmed.
Special handling	Character↔token alignment built from the BPE (whitespace stripped from token edges); optional punctuation splitting so spans like "Ltd.," are reachable; a round-trip check gold → tags → spans gives the score ceiling; 512-token sliding windows (stride 384), predictions averaged with edge-distance weights; bidirectional mask also for A and D (causal kept as an ablation).
What is different	The only **span/token-level** task. Errors are about exact boundaries, not just labels. Macro averaging makes rare classes (WIT, A.COUNSEL) as important as frequent ones.
Experiments	E1 C full FT · E2 B-encoder **LoRMA** · E3 A_v2 LoRA (bidirectional; E3b causal) · E4 D adapters (E4b BitFit) · E5 **KD** E1 → 4-layer C (E5c same student without KD; E5b → B) · E6 C + CRF (E6b class weights) · extras: frozen probe, partial FT, prompt tuning, LoRMAπ.

Phase 3: Rhetorical Role prediction (RR) [Planned]

Data	Judgments as ordered sentence lists, two domains: Corporate Law (CL) and IT law. Train + dev + test per domain; label vocabulary file.
Input → output	Sentence list → one rhetorical role per sentence (facts, issue, arguments, precedent, statute, ruling, …).
Metric	Macro-F1 over roles.
Task module	**Sentence level, hierarchical.** Tokens of each sentence → mean-pooled sentence vector (or a [sep]-token vector) → a small context layer over the sentence sequence (BiLSTM or 1–2 transformer layers; optional CRF over roles) → Linear(d → #roles). CE per sentence.
What is different	Context *between* sentences matters as much as the sentence itself (a ruling comes at the end, facts at the start), so a position feature or sentence-sequence layer helps. Two domains: train jointly (more data) vs. per domain vs. joint + domain tag. Strong label imbalance across roles.
Techniques	Same menu as Phase 2 (≥3 incl. LoRMA + KD). KD on per-sentence role distributions.

Phase 4: Court Judgment Prediction (CJPE) [Planned]

Data	Case texts with binary outcome; train + dev + test (train merges the original single- and multi-instance sets).
Input → output	Case text → 0 / 1.
Metric	Macro-F1 (precision and recall as secondary).
Task module	**Document level.** Each 512-token window → pooled vector (mean or first token); windows combined by mean / max / attention pooling into one document vector → Linear(d → 2). CE, class-weighted if the outcomes are imbalanced.
What is different	One label per **very long** document, so the key design choice is how window evidence is aggregated (and which part of the case to keep if truncating). The answer depends on reasoning, not on spotting tokens, so it is the hardest test of whether Phase-1 representations transfer.
Techniques	≥3 incl. LoRMA + KD; KD on the 2-class document distribution.

Phase 5: Legal Statute Identification (LSI) [Planned]

Data	Case texts with lists of statute IDs; train + dev + test.
Input → output	Case text → set of statute IDs (multi-label, e.g. [1, 7, 23]).
Metric	Macro-F1 (micro-F1 and subset accuracy secondary).
Task module	Same document pooling as Phase 4, then Linear(d → #statutes) with a **sigmoid per statute** and BCE loss (optionally focal / asymmetric loss for rare statutes). Decision thresholds tuned on dev (global or per statute).
What is different	**Multi-label**: each statute is an independent yes/no, unlike the softmax of Phases 2–4. Long-tailed label distribution, so macro-F1 rewards getting rare statutes right; threshold choice matters as much as the model.
Techniques	≥3 incl. LoRMA + KD; KD on per-statute sigmoid probabilities.

Phase 6: Abstractive Summarisation (current window: 27 Oct – 2 Nov)

[Planned]

Data	Documents paired with reference summaries (train) + test documents. No dev split: hold out part of train.
Input → output	Legal document → generated summary text.
Metric	BERTScore F1 (ROUGE-1 / ROUGE-L secondary).
Task module	**Generation**, so the LM head comes back. B: encoder reads the document, decoder writes the summary (natural seq2seq). C: prefix-LM: document as bidirectional prefix, summary generated causally (matches its pretraining). A / D: "document <sep> summary" with the loss only on summary tokens. Next-token CE; beam search or sampling with length control.
What is different	Output is free text, not labels. Inputs are far longer than the context, so a long-input strategy is needed (chunk-and-merge, or extract key sentences first, then abstract; D's long window helps). Decoding settings (beam size, repetition penalty, length) change the score a lot.
Techniques	≥3 incl. LoRMA + KD. Prompt / prefix tuning fit generation naturally. KD as **sequence-level KD** (student trained on teacher-generated summaries) or word-level KL on next-token distributions.

Phase 7: Prior Case Retrieval (PCR) [Planned]

Data	Query cases with relevant candidate IDs; each split (train/dev/test) has its own candidate pool; texts split into sentences.
Input → output	Query case + candidate pool → 7 unique candidate IDs ranked by relevance.
Metric	Micro-F1@7 against the full gold set.
Task module	**Embedding / ranking.** Bi-encoder: query and candidate each → pooled document embedding (optional projection) → cosine similarity → top 7. Trained with a contrastive loss (InfoNCE): relevant candidates pulled close, in-batch and **hard negatives** from the pool pushed away. Optionally fuse with a lexical (TF-IDF/BM25) score computed from the data itself.
What is different	No classifier; the output space is a candidate pool. Quality depends on pooling, negatives and the similarity objective. Precedents are relevant for legal reasons, not just word overlap.
Techniques	≥3 incl. LoRMA + KD. KD by matching the student's query–candidate similarity distribution to the teacher's.

Task modules side by side

With backbone output H = one d-dimensional vector per token, each phase reads H differently:

Phase	Read H at	Head	Loss	Decoding / output
2 NER	every token	Linear d→25 or CRF	token CE / CRF NLL	Viterbi → character spans
3 RR	every sentence (+ sentence context)	Linear d→#roles (+ CRF)	CE per sentence	label per sentence
4 CJPE	whole document	Linear d→2	CE	argmax → 0/1
5 LSI	whole document	Linear d→#statutes, sigmoid	BCE	threshold → set of IDs
6 Summ.	– (generate)	LM head / decoder	next-token CE	beam / sampling → text
7 PCR	whole document (embedding)	projection (optional)	contrastive (InfoNCE)	cosine → top-7 IDs

Toy example. A 3-token window "Supreme Court dismissed" gives H = [[0.2, 0.9], [0.1, 0.8], [0.7, 0.3]]. NER applies the head to each row (B-COURT, I-COURT, O). CJPE mean-pools to [0.33, 0.67] and outputs one label. LSI uses the same pooled vector with independent sigmoids, so several statutes can be "on". PCR uses the pooled vector as the case embedding and compares it with cosine similarity.

4. Fine-tuning techniques

Notation: a frozen pretrained linear layer W_0 ($d_{out} \times d_{in}$) computes $h = W_0x$. Parameter counts are for **one** 640 × 640 layer (d = 640, as in C and D) so the techniques can be compared directly: the full layer has 409,600 weights. All PEFT methods are initialised so the model starts **exactly** as the pretrained one (verified in `tests/test_pipeline.py`). The task head is always trained.

Full fine-tuning

What it does	Every backbone weight is updated together with the head, at a small learning rate for the backbone (e.g. 5e-5) and a larger one for the new head.
How (formula)	$W_0 \leftarrow W_0 - \eta\nabla L$ for all layers.
Trainable params (one 640×640 layer)	409,600 of 409,600 (100 %)
What is different	The reference point: usually the most accurate, but it needs the most memory and a full model copy per task, and it can overfit small data. Optional **layer-wise LR decay** (LLRD) gives lower layers smaller learning rates.
Used in Phase 2	E1, E6 (C); KD students

Partial / layer-wise fine-tuning

What it does	Only the top k blocks (and the final norm) are trained; the lower blocks stay frozen.
How (formula)	Gradients flow only through the last k layers.
Trainable params (one 640×640 layer)	a whole layer for each trained block, 0 for frozen ones (e.g. k = 4 of 11)
What is different	Coarse-grained: assumes lower layers hold general language features and upper layers task features. Cheaper backward pass. A middle ground between full FT and PEFT.
Used in Phase 2	X_A2_partial4

Frozen backbone (linear probe)

What it does	Backbone completely frozen; only the head (optionally a small MLP) is trained.
How (formula)	$h = W_0x$, no update.
Trainable params (one 640×640 layer)	0
What is different	Not meant to win: it **measures how much task information the Phase-1 representations already contain**. Useful evidence for the "does pretraining transfer?" analysis.
Used in Phase 2	X_C_frozen_probe

LoRA (Hu et al., 2021)

What it does	Adds a trainable low-rank **additive** update beside each frozen linear layer.
How (formula)	$h = W_0x + (\alpha/r)\cdot B\,A\,x$, with A: $r \times d_{in}$ (random), B: $d_{out} \times r$ (zeros), $r = 8$, $\alpha = 16$.
Trainable params (one 640×640 layer)	$r(d_{in} + d_{out}) = 8 \times 1280 = 10{,}240$ (2.5 %)
What is different	Additive: the update $\Delta W = BA$ has rank $\leq r$. Cheap, can be merged into W_0 after training (no inference cost). The standard PEFT baseline.
Used in Phase 2	E3, E3b (A_v2)

LoRMA (Bihany, Patel & Modi, 2025): compulsory

What it does	Low-rank **multiplicative** adaptation: the pretrained output is *transformed* by a matrix instead of having something added to it.
How (formula)	LoRMA_+: $h = (I + (\alpha/r)\cdot B\,A)\,W_0x$, with A: $r \times d_{out}$, B: $d_{out} \times r = 0$. Computed right-to-left, $B(A(W_0x))$, so the $d \times d$ matrix is never formed. LoRMA_π: $h = R_\pi(BA)\,W_0x$, where row i of BA is rotated cyclically by i (rank inflation to full rank); initialised so $R_\pi(BA) = I$.
Trainable params (one 640×640 layer)	$2\,r\,d_{out} = 10{,}240$ (same as LoRA for square layers)
What is different	Unlike LoRA, the update $\Delta W = (\alpha/r)\cdot BA\cdot W_0$ is **tied to the pretrained weights**: it rescales and mixes W_0's output directions. The identity term (LoRMA_+) or the rotation (LoRMA_π) avoids a rank-r bottleneck on W_0 itself. LoRMA_π costs a $d \times d$ matmul per token, so it is used on attention layers only.
Used in Phase 2	E2 (B encoder, LoRMA+), E2b (C, LoRMAπ)

Houlsby adapters (Houlsby et al., 2019)

What it does	A small bottleneck MLP inserted after the attention output and after the FFN output of every block.
How (formula)	$y = W_0x$; $y \leftarrow y + U\cdot\text{GELU}(D\cdot y)$, with D: $64 \times d$ (down), U: $d \times 64$ (up, zeros).
Trainable params (one 640×640 layer)	$2\cdot64\cdot640 + 64 + 640 = 82{,}624$ per adapter (2 per block)
What is different	Adds **new sequential modules** (extra depth and non-linearity) instead of modifying existing weights, so it slightly increases inference cost and cannot be merged away. More expressive per parameter than BitFit.
Used in Phase 2	E4 (D)

BitFit (Ben Zaken et al., 2022)

What it does	Train only bias terms (plus normalisation gains). Our Phase-1 linears have no biases, so a zero-initialised bias is **added** to every block linear and only those are trained.
How (formula)	$h = W_0x + b$, only b (and RMSNorm weights) trainable.
Trainable params (one 640×640 layer)	640 (0.16 %)
What is different	The most extreme sparse method: it shifts activations but cannot change how features are mixed. A lower bound for PEFT capacity.
Used in Phase 2	E4b (D)

Prompt tuning (Lester et al., 2021)

What it does	n learned "virtual token" vectors are prepended to the input embeddings; the whole backbone stays frozen.
How (formula)	input = $[p_1 \ldots p_n; \text{embed}(x)]$, $n = 20$.
Trainable params (one 640×640 layer)	$n\cdot d = 12{,}800$ for the whole model (not per layer)
What is different	Changes only the **input**, not any weight. The smallest footprint, but the weakest for small models; most natural for generation (Phase 6). Prefix tuning is the per-layer variant (learned key/value prefixes).
Used in Phase 2	X_C_prompt

Knowledge distillation (Hinton et al., 2015): compulsory

What it does	A trained large **teacher** produces soft predictions; a smaller **student** learns from both the gold labels and the teacher's distribution.
How (formula)	$L = \alpha \cdot CE(\text{student, gold}) + (1-\alpha) \cdot T^2 \cdot KL(\text{softmax}(z_t/T) \parallel \text{softmax}(z_s/T))$, $\alpha = 0.5$, $T = 2$. Teacher scores are computed once and cached.
Trainable params (one 640×640 layer)	Student-dependent: here a 4-layer C (≈ 40 % of C's blocks) or B's 42 M-parameter encoder
What is different	Not a way of adapting weights but a way of **shrinking** the model: soft labels carry "dark knowledge" (e.g. 70 % APP / 25 % RESP) that hard labels lose. It is judged on the accuracy–size–speed trade-off, hence the same student without KD (E5c) is needed as a control.
Used in Phase 2	E5, E5c (control), E5b

Comparison at a glance

Technique	Changes	Trainable (per 640^2 layer)	Merge into W_0?	Extra inference cost	Main strength / risk
Full FT	all weights	409,600	–	none	most accurate / memory, overfitting
Partial FT	top-k blocks	per trained block	–	none	cheap / lower layers frozen
LoRA	W + BA (additive)	10,240	yes	none after merge	robust baseline / rank-r update
LoRMA+	(I + BA)·W (multiplicative)	10,240	yes	none after merge	update tied to W_0 / new, less studied
LoRMAπ	Rπ(BA)·W	10,240	yes	d^2 matmul unless merged	full-rank update / costly per token
Adapters	new bottleneck modules	82,624 per adapter	no	small	expressive / adds depth
BitFit	biases only	640	yes	none	tiny / limited capacity
Prompt tuning	input embeddings	12,800 total	–	n extra tokens	smallest / weak on small models
KD	student size	student's	–	**less** (smaller model)	efficiency / needs a good teacher

5. Technique × phase plan

Each phase must use ≥3 techniques including LoRMA, plus one KD experiment, spread over all four Phase-1 models. Phase 2 is the executed plan; for later phases the cells are suggestions that satisfy the rules and fit the task.

Technique	P2 NER	P3 RR	P4 CJPE	P5 LSI	P6 Summ.	P7 PCR
Full fine-tuning	● C	●	●	●	● (B)	●
LoRMA (compulsory)	● B	●	●	●	●	●
LoRA	● A_v2	●	●	●	●	●
Adapters / BitFit	● D	○	○	○	○	○
Prompt / prefix tuning	○	○	○	○	●	○
Partial FT / frozen probe	○	○	○	○	○	○
Knowledge distillation (compulsory)	● C→C-4L	●	●	●	● (seq-level)	● (similarity)
Special head / loss	CRF, class weights	sentence context + CRF	window pooling	sigmoid + BCE, thresholds	LM head, decoding	contrastive, hard negatives

● = main experiment, ○ = optional ablation.

6. Where the code lives

File	Contents
`p2lner/tokenizer.py`	Phase-1 BPE (load only) + per-token character offsets + disk cache
`p2lner/backbones.py`	Phase-1 checkpoint → token encoder (causal / bidirectional, padding masks, fused attention paths)
`p2lner/peft.py`	full, partial, frozen, LoRA, LoRMA+ / LoRMAπ, Houlsby adapters, BitFit, prompt tuning
`p2lner/heads.py, model.py`	linear head, CRF; CE / class-weighted / focal / KD losses
`p2lner/spans.py, metrics.py`	BIO conversion, constrained Viterbi, boundary trimming, strict P/R/F1, error analysis
`p2lner/train.py`	sliding windows, length-bucketed training, early stopping, teacher cache, refit, submission zip
`CS781_Phase2_LNER.ipynb / _v2.ipynb`	single-file Kaggle notebooks (v2: two GPUs in parallel)
`tests/test_pipeline.py`	checks: offsets, round trip, backbone equivalence, padding invariance, PEFT identity at init, CRF vs. brute force, end-to-end run

References: IL-TUR benchmark (arXiv:2407.05399) · Hu et al. 2021, LoRA · Bihany, Patel & Modi 2025, LoRMA (arXiv:2506.07621, ACL Findings) · Houlsby et al. 2019, Adapters · Ben Zaken et al. 2022, BitFit · Lester et al. 2021, Prompt tuning · Li & Liang 2021, Prefix tuning · Hinton et al. 2015, Knowledge distillation · Lafferty et al. 2001, CRF.